

Build
Design
Scale
Succeed

Think
Design
Architect
Explain
Get Hired



SYSTEM DESIGN INTERVIEW HANDBOOK

Real
Scenarios
Real Solutions

From Architecture Fundamentals to
Production-Scale AWS Design



Compute

EC2 | ECS | EKS
Lambda | Auto Scaling



Storage

S3 | EBS | EFS
Glacier



Databases

RDS | Aurora
DynamoDB | ElastiCache



Networking

VPC | Route 53
ALB | CloudFront



Security

IAM | KMS | WAF
Shield | Secrets Manager



Observability

CloudWatch | CloudTrail
X-Ray | Logs | Metrics



Messaging

SQS | SNS | EventBridge
Kinesis | Step Functions



Application Services

API Gateway | CloudFront
Elastic Beanstalk | Amplify



Reliability

Multi-AZ | Auto Scaling
Backup | Disaster Recovery



Design Skills

Scalability | Trade-offs
Cost | Real-world Scenarios



PRACTICAL EXPLANATIONS
REAL INTERVIEW SCENARIOS
ARCHITECTURE DIAGRAMS
EXPERT DESIGN STRATEGIES
STEP-BY-STEP THINKING FRAMEWORKS

Better
Engineers
Brighter
Careers

Think
Architect
Not Just
Services



Good
Design
Solves
Real Problems

HOW TO APPROACH AN AWS SYSTEM DESIGN INTERVIEW

1 WHAT SYSTEM DESIGN INTERVIEWS ACTUALLY TEST



- Your ability to solve real-world problems
- Architectural thinking
- Trade-offs and decision making
- Communication skills
- Understanding of scalability, reliability, security and cost
- Not just memorizing AWS services

2 FUNCTIONAL vs NON-FUNCTIONAL REQUIREMENTS

Functional Requirements (What the system should do)

- Key features
- User actions
- Business logic
- Data to store and retrieve
- Core behaviour

Non-Functional Requirements (How the system should perform)

- Scalability
- High availability
- Low latency
- Security
- Durability
- Cost efficiency
- Compliance

3 CLARIFYING QUESTIONS



- What problem are we solving?
- Who are the users?
- What are the key features?
- What is the expected scale?
- What are the latency and availability requirements?
- Are there any business or technical constraints?

4 USERS, TRAFFIC, DATA, LATENCY, AVAILABILITY



- Users: How many users? Where are they located?
- Traffic: Requests per second (RPS)? Peak traffic?
- Data: How much data? What type? Growth rate?
- Latency: What is the acceptable response time?
- Availability: Uptime requirements (e.g. 99.9%)?

5 CONSTRAINTS AND ASSUMPTIONS



- Budget or cost limits
- Technology preferences or restrictions
- Timeline
- Existing systems to integrate with
- Regulatory or compliance requirements
- State your assumptions clearly if information is missing

6 AVOID THE COMMON MISTAKE



Do not start by naming AWS services immediately.

- First understand the problem
- Clarify requirements
- Think about the architecture
- Then choose the right AWS services
- Show your reasoning

"It's not about which services you know, but how well you solve the problem."

7 SYSTEM DESIGN FRAMEWORK



Learn
Build
Grow

@veriqta

veriqta

veriqta

veriqta

@veriqta

@veriqta

VERIQTA

TECH KNOWLEDGE FOR A BRIGHTER TOMORROW

Same
Better
Stronger

ESTIMATING SCALE BEFORE DESIGNING

Good estimates help you design the right architecture, avoid over-engineering, and make trade-offs like a real engineer.

1 DAILY AND MONTHLY ACTIVE USERS



- DAU: Unique users per day
- MAU: Unique users per month
- Helps you understand user scale and growth
- Not all users are active every day

Example:
1M MAU, 200K DAU

2 REQUESTS PER SECOND (RPS)



- Estimate total requests
- Divide by seconds in a day
- Consider peak traffic
- Include all types of requests (read, write, APIs, static)

Example:
100 million requests/day
= $100,000,000 \div 86,400 \approx 1,157$ RPS

3 READ/WRITE RATIOS



Read
Write

- Understand your workload
- Read-heavy (e.g. content, social)
- Write-heavy (e.g. payments, orders)
- Affects database choice, caching strategy, and scaling

Example:
80% reads, 20% writes

4 PEAK vs AVERAGE TRAFFIC



- Peak traffic can be 2-10x higher than average
- Design for peak, but optimize for average
- Consider time zones, seasonality, and events
- Use auto scaling and load balancing

5 STORAGE GROWTH



- Estimate data per user
- Multiply by total users
- Factor in growth rate
- Consider backups and replicas

Example:
1 GB per user $\times$ 1M users
= 1 PB (before replication/backup)

6 BANDWIDTH



- Estimate data transferred in and out
- Consider users, media files, APIs, and downloads
- Account for global users
- Influences CDN, networking, and cost

Example:
1,000 RPS $\times$ 100 KB response
= 100 MB/s $\approx$ 800 Mbps

7 SIMPLE BACK-OF-THE-ENVELOPE CALCULATIONS



- Start with round numbers
- Make reasonable assumptions
- Break the problem into parts
- Estimate, don't overcomplicate
- Show your thinking clearly

8 WHY ESTIMATES INFLUENCE ARCHITECTURE



- Helps you choose the right services
- Determines if you need caching
- Influences database type and size
- Affects scaling, high availability, and disaster recovery
- Helps with cost estimation
- Shows you can think like a real engineer

9 INTERVIEW PRACTICE CALCULATION



- Design a simple application (e.g. photo sharing app)
- Estimate users, RPS, storage, and bandwidth
- Explain your assumptions
- Discuss peak traffic
- Show how your estimates guide the architecture



KEY TAKEAWAY

You don't need perfect numbers. You need reasonable estimates and clear thinking. Explain your assumptions and be ready to adjust based on the interviewer's input.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Better
Engineers
Brighter
Futures

VERIQTA

LEARN | BUILD | GROW

Knowledge
Into
Opportunity

Same
Better
Stronger

AWS ARCHITECTURE BUILDING BLOCKS

Learn
Design
Build
Grow

Core AWS services you need to know for system design interviews.
Understand what each service does and how they work together.

1 REGIONS



- A geographic area in the world.
- Each region is independent.
- Choose a region close to your users.

Example: us-east-1 (N. Virginia), eu-west-1 (Ireland)

2 AVAILABILITY ZONES



- Multiple isolated data centers within a region.
- Designed for high availability.
- Use multiple AZs for fault tolerance.

Example: us-east-1a, us-east-1b, us-east-1c

3 VPCs



- Your private network in AWS.
- Control IP ranges, subnets, routing, and security.
- Isolate and protect your resources.

Example: 10.0.0.0/16

4 PUBLIC AND PRIVATE SUBNETS



- Public subnet: Resources with internet access (e.g. ALB, NAT).
- Private subnet: Resources without direct internet access (e.g. EC2, RDS).
- Use multiple AZs for resilience.

5 EC2



- Virtual servers in the cloud.
- Run your applications.
- Choose the right instance type for your workload.
- Can be used with Auto Scaling and Load Balancers.

6 ELASTIC LOAD BALANCING (ELB)



- Distributes traffic across multiple targets (EC2, ECS, etc).
- Improves availability and scalability.
- Types: ALB (HTTP/HTTPS), NLB (TCP/UDP), CLB (legacy).

7 AUTO SCALING



- Automatically increases or decreases EC2 instances based on demand.
- Helps handle traffic spikes.
- Keeps your application highly available and cost-efficient.

8 S3



- Object storage for files, images, backups, logs, etc.
- Highly durable and scalable.
- Common use cases: static websites, media files, backups, data lakes.
- Pay only for what you use.

9 RDS



- Managed relational database (MySQL, PostgreSQL, etc).
- Automatic backups, patching, and high availability.
- Supports Multi-AZ and read replicas.
- Focus on your application, not database administration.

10 DYNAMODB



- Fully managed NoSQL database.
- Highly scalable and low latency.
- Great for high-traffic applications.
- Automatically scales to handle billions of requests.

11 ROUTE 53



- Scalable DNS service.
- Routes users to your applications.
- Supports health checks, failover, and latency-based routing.
- Helps you build highly available applications.

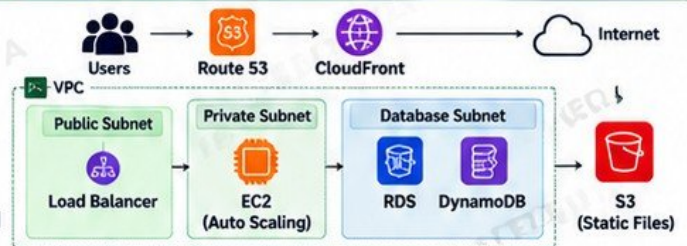
12 CLOUDFRONT



- Content Delivery Network (CDN).
- Caches your content at edge locations globally.
- Reduces latency for users.
- Commonly used with S3, ALB, and static websites.

13 HOW THESE PIECES FORM A SYSTEM

- Users access your application through Route 53 and CloudFront.
- Traffic is routed to a Load Balancer in public subnets.
- Applications run on EC2 instances (Auto Scaling) in private subnets.
- Data is stored in RDS or DynamoDB.
- Static files are stored in S3 and delivered via CloudFront.
- All of this runs inside a VPC across multiple Availability Zones and a region.



Better
Engineers
Futures

@veriqta

veriqta

veriqta

veriqta

@veriqta

@veriqta

VERIQTA

LEARN | BUILD | GROW

Knowledge
Into
Opportunity

Better
Engineers
Brighter
Futures

DESIGNING THE NETWORK FOUNDATION

Plan
Secure
Scale
Succeed

A well-designed network is the foundation of a secure, scalable and highly available AWS architecture.

1 VPC DESIGN



- Your isolated network in AWS.
- Provides control over IP ranges, subnets, routing and security.
- Foundation for all your network resources.

2 CIDR PLANNING



- Choose an IP range for your VPC (e.g. 10.0.0.0/16).
- Plan subnets for each tier (public, private app, private DB).
- Leave room for growth.
- Avoid overlapping CIDRs with on-prem or other VPCs.

3 PUBLIC SUBNETS



- Resources with internet access (e.g. ALB, NAT Gateway).
- Route to Internet Gateway (IGW).
- Typically used in multiple Availability Zones.
- Keep them minimal and secure.

4 PRIVATE APPLICATION SUBNETS



- Host application servers (e.g. EC2, ECS, EKS).
- No direct internet access.
- Outbound internet access via NAT Gateway.
- Distribute across multiple AZs for high availability.

5 PRIVATE DATABASE SUBNETS



- Host databases (e.g. RDS).
- No direct internet access.
- Accessible only from application subnets.
- Use multiple AZs for high availability.
- Apply strict security controls (Security Groups, NACLs).

6 ROUTE TABLES



- Control where network traffic goes.
- Public subnets route to Internet Gateway.
- Private subnets route to NAT Gateway (for internet access) or stay internal.
- Each subnet is associated with a route table.

7 INTERNET GATEWAY (IGW)



- Connects your VPC to the internet.
- Enables inbound and outbound traffic for public subnets.
- Required for resources like ALB, bastion hosts, or public-facing services.

8 NAT GATEWAY



- Allows private subnets to access the internet (for updates, package downloads, etc.).
- No inbound connections from the internet.
- Deploy in multiple AZs for high availability.

9 SECURITY GROUPS



- Act as a virtual firewall for your resources.
- Control inbound and outbound traffic.
- Stateful (return traffic is automatically allowed).
- Use least privilege (only allow required ports and sources).
- Example: Allow HTTPS (443) from ALB to EC2 instances.

10 MULTI-AZ NETWORK DESIGN



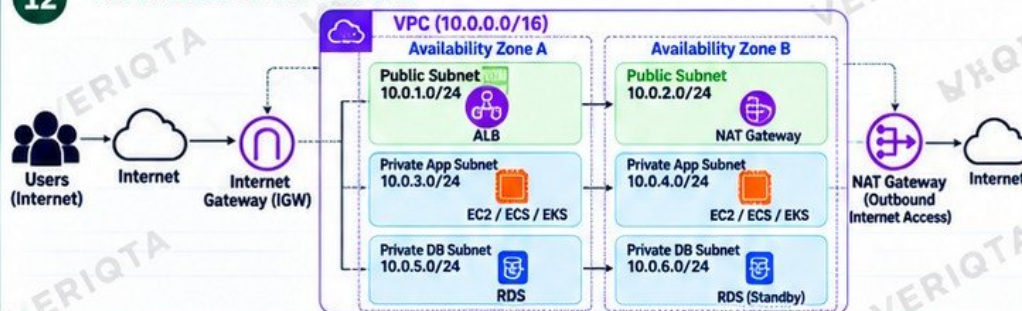
- Use at least 2 Availability Zones.
- Place public, private app, and private DB subnets in each AZ.
- Improves availability and fault tolerance.
- Ensures your application continues to run even if one AZ fails.

11 TRAFFIC-FLOW REASONING



- Understand how traffic flows end-to-end.
- Map inbound (user → ALB) and outbound (private → internet). Consider internal traffic between tiers (ALB → app → DB).
- Identify where security controls apply.
- Be able to explain each hop during the interview.

12 REFERENCE ARCHITECTURE



KEY TAKEAWAYS

- ✓ Design your VPC and subnets with a clear purpose.
- ✓ Use multiple AZs.
- ✓ Control traffic with route tables and security groups.
- ✓ Keep private resources private.
- ✓ Understand and explain traffic flow.
- ✓ Always design for security, high availability and scale.

Knowledge
Into
Opportunity

@veriqta

veriqta

veriqta

veriqta

@veriqta

@veriqta

VERIQTA

LEARN | BUILD | GROW

Same
Better
Stronger

Better
Engineers
Brighter
Futures

DNS, TRAFFIC ROUTING, AND EDGE ARCHITECTURE

Design
Scale
Deliver
Opportunities

Get users to your application from anywhere in the world, securely, reliably and with the best performance using AWS global infrastructure.

1 ROUTE 53



- Scalable DNS service.
- Register domain names.
- Route traffic to AWS resources and external resources.
- Supports health checks and failover.
- Routing policies: Simple, Weighted, Latency-based, Failover, Geolocation, Multi-value.

Example:
Route users to the closest region using latency-based routing.

2 DNS RESOLUTION



example.com → IP address

- User enters a domain name (e.g. www.example.com).
- DNS translates the domain name to an IP address.
- Route 53 returns the best IP based on your routing policy.
- The user's device connects to that IP address.

Example:
www.example.com → 13.32.45.67

3 CLOUDFRONT



- Global Content Delivery Network (CDN).
- Caches content at edge locations worldwide.
- Reduces latency and improves performance.
- Commonly used for static content (images, videos, CSS, JS) and dynamic content (with caching).
- Works with S3, ALB, and custom origins.

Example:
Serve website static files from S3 through CloudFront.

4 AWS GLOBAL ACCELERATOR



- Improves global application performance using AWS global network.
- Provides static anycast IP addresses.
- Routes traffic to the nearest healthy AWS region.
- Ideal for latency-sensitive applications (e.g. gaming, real-time apps).

Example:
Users connect to the nearest AWS edge location, then routed to your application.

5 APPLICATION LOAD BALANCER (ALB)



- Operates at Layer 7 (HTTP/HTTPS).
- Routes traffic based on content (path, host, headers).
- Works with multiple target groups (EC2, ECS, EKS, Lambda).
- Supports TLS termination, health checks and auto scaling.

Example:
Route /api to backend service and / to frontend service.

6 NETWORK LOAD BALANCER (NLB)



- Operates at Layer 4 (TCP/UDP).
- Handles millions of requests with ultra-low latency.
- Ideal for non-HTTP traffic (e.g. databases, gaming, streaming).
- Supports static IP addresses and TLS pass-through.

Example:
Load balance TCP traffic to your application servers.

7 TLS TERMINATION



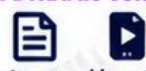
- Decrypts HTTPS traffic at the edge (CloudFront), load balancer (ALB/NLB) or other services.
- Improves security and offloads encryption from your application servers.
- Use AWS Certificate Manager (ACM) to manage SSL/TLS certificates.
- You can terminate at CloudFront, ALB or NLB depending on your architecture.

8 ROUTING USERS TO APPLICATIONS



- Use Route 53 to direct users to the right endpoint (e.g. CloudFront, ALB, NLB).
- CloudFront routes to your origin (S3, ALB, etc.).
- ALB/NLB distribute traffic to healthy targets.
- Use routing policies for high availability and disaster recovery.

9 STATIC vs DYNAMIC CONTENT



- Static content: Images, videos, CSS, JS, files. Best served from S3 + CloudFront.
- Dynamic content: User-specific or real-time data (APIs, dashboards). Best served through ALB (often behind CloudFront).
- You can cache some dynamic content at the edge to improve performance.

10 CHOOSING THE CORRECT TRAFFIC LAYER



- Use Route 53 for DNS and global routing.
- Use CloudFront for performance and global content delivery.
- Use Global Accelerator for ultra-low latency and static anycast IP.
- Use ALB for HTTP/HTTPS traffic and advanced routing.
- Use NLB for high-performance TCP/UDP traffic.
- Consider security, latency, cost and application requirements.

TYPICAL EDGE ARCHITECTURE



Knowledge
Into
Opportunity

@veriqta

veriqta

veriqta

veriqta

@veriqta

@veriqta

VERI QTA
LEARN | BUILD | GROW

Same
Better
Stronger

COMPUTE ARCHITECTURE AND HORIZONTAL SCALING

Choose the right compute for your workload, design for scale, and keep your application highly available.

1 EC2 (ELASTIC COMPUTE CLOUD)



- Virtual machines in the cloud.
- Full control over OS and setup.
- Use for: custom applications, legacy systems, specific software requirements.
- Works with Auto Scaling, Load Balancers, and other services.

2 AUTO SCALING GROUPS



- Automatically add or remove EC2 instances.
- Maintain desired capacity.
- Scale based on metrics (CPU, memory, custom).
- Use with Launch Templates and Load Balancers.

3 ECS (ELASTIC CONTAINER SERVICE)



- Run containerized applications without managing Kubernetes.
- Integrates with ECR, ALB, and Auto Scaling.
- Good for microservices and simpler container needs.

4 EKS (ELASTIC KUBERNETES SERVICE)



- Managed Kubernetes on AWS.
- Best for complex, large-scale containerized applications.
- Full Kubernetes features with AWS integration (IAM, VPC, logging, etc).

5 AWS LAMBDA



- Serverless compute.
- Run code without managing servers.
- Automatically scales from zero to thousands of concurrent executions.
- Use for event-driven workloads, APIs, and background jobs.

6 HORIZONTAL vs VERTICAL SCALING

HORIZONTAL SCALING (Scale out)



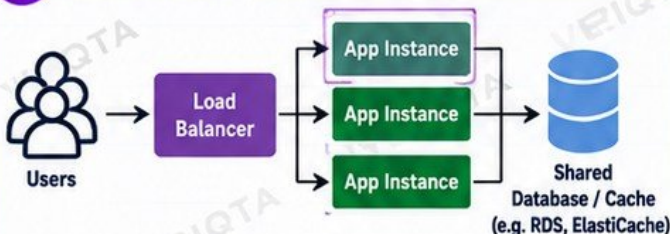
- Add more instances.
- Handles higher traffic.
- More fault tolerant.
- Common in cloud architecture.

VERTICAL SCALING (Scale up)



- Increase instance size (CPU, memory).
- Simple but has limits.
- Not as fault tolerant.
- Used for specific workloads.

7 STATELESS APPLICATION DESIGN



- Keep application state out of the instances.
- Use a shared database or cache.
- Enables horizontal scaling and faster recovery.
- Works well with Auto Scaling, ECS, EKS, and Lambda.

8 HEALTH CHECKS



- Check if instances or containers are healthy.
- Used by Load Balancers and Auto Scaling Groups.
- Detect and remove unhealthy instances.
- Implement application-level health endpoints (e.g. /health).
- Helps maintain high availability.

9 SCALING POLICIES



- Scale based on metrics like CPU, memory, or request count.
- Types: target tracking, step scaling, scheduled scaling.
- Set minimum, desired, and maximum capacity.
- Use CloudWatch metrics and alarms.
- Choose policies based on workload patterns (predictable or unpredictable).

10 CHOOSING COMPUTE BASED ON WORKLOAD

Workload Type	Recommended Service(s)	Why
Simple web application	EC2 + Auto Scaling	Full control, flexible, widely used
Containerized application (simple)	ECS	Easier to manage than Kubernetes
Containerized application (complex)	EKS	Full Kubernetes features
Event-driven / unpredictable traffic	Lambda	Serverless, scales automatically
Batch processing / background jobs	EC2, ECS, Lambda	Depends on runtime and duration
Microservices at scale	EKS (or ECS)	Better orchestration and scalability
Legacy application	EC2	Full control over environment



KEY TAKEAWAYS

- ✓ Understand your workload requirements.
- ✓ Design for horizontal scaling.
- ✓ Keep your application stateless.
- ✓ Use health checks and proper scaling policies.
- ✓ Choose the right compute service, there is no one-size-fits-all.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

SAME
BETTER
STRONGER

CHOOSING THE RIGHT DATABASE

Understand the options, match the database to the problem, and explain your choice with confidence.

1 RDS (RELATIONAL DATABASE SERVICE)



- Managed relational database.
- Supports MySQL, PostgreSQL, MariaDB, Oracle, SQL Server.
- Easy to set up, operate, and scale.
- Good for traditional applications with structured data.
- Use read replicas for read scaling.

2 AURORA



- AWS optimized relational database (MySQL and PostgreSQL compatible).
- Higher performance and availability than standard RDS.
- Automatic scaling of storage.
- Multi-AZ by default.
- Good for high-traffic applications.

3 DYNAMODB



- Managed NoSQL database.
- Serverless and highly scalable.
- Single-digit millisecond latency.
- Key-value and document data.
- Good for large-scale, high-throughput applications.
- No complex schema, scales automatically.

4 ELASTICACHE



- Managed in-memory caching.
- Supports Redis and Memcached.
- Used to improve application performance.
- Great for caching database queries, sessions, and frequently accessed data.
- Not a primary database.

5 DOCUMENTDB



- Managed NoSQL database (MongoDB compatible).
- Stores and queries JSON documents.
- Good for content, catalog, and user data.
- Automatically scales storage.
- Use when you need MongoDB compatibility without self-managing.

6 OPENSEARCH



- Managed search and analytics engine (successor to Elasticsearch).
- Used for full-text search, log analytics, and data exploration.
- Real-time search and filtering.
- Good for observability, application search, and large data sets.
- Not a general-purpose database.

7 RELATIONAL vs NOSQL

Relational (SQL)	NoSQL
• Structured data • Fixed schema • Strong consistency • Use SQL • Examples: RDS, Aurora	• Flexible schema • Handles large scale • Different data models • Eventual or strong consistency (depends) • Examples: DynamoDB, DocumentDB, OpenSearch

8 ACCESS PATTERNS



- How will the data be accessed?
- Read-heavy or write-heavy?
- Simple key lookups?
- Complex queries?
- Need for full-text search?
- Time-series data?
- Relationships between data?
- Access patterns help determine the right database.

9 CONSISTENCY



- Strong consistency: Always returns the latest data.
- Eventual consistency: Data is updated asynchronously.
- Choose based on application requirements.
- DynamoDB can be configured for strong or eventual consistency.
- Understand the trade-offs.

10 QUERY REQUIREMENTS



- Simple queries (key-value lookups)
- Complex queries (joins, aggregations)
- Full-text search
- Geospatial queries
- Real-time analytics
- These requirements influence the database choice.

11 DATABASE SELECTION INTERVIEW FRAMEWORK



- ✓ Start with the problem, not a specific AWS service.
- ✓ Use requirements, access patterns, scale, and constraints to evaluate options.
- ✓ Explain your reasoning and trade-offs clearly.
- ✓ There is usually more than one correct answer.



KEY TAKEAWAY

The best engineers choose the right database for the problem, not the most popular one.

"It's not about knowing all the databases, but knowing when and why to use each one."

— VERIQTA



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

SAME
BETTER
STRONGER

DESIGNING THE DATA LAYER FOR SCALE

Design a data layer that is reliable, scalable, and ready for real-world traffic.

Data
Drives
Better
Systems

1 PRIMARY KEYS



- Uniquely identify each record in a table.
- Used for fast lookups and efficient data access.
- Should be immutable (not changed).
- Choose keys that are unique, meaningful, and scalable.

EXAMPLE (RDS)

user_id (PK), order_id (PK)

2 INDEXES



- Speed up data retrieval for specific queries.
- Improve read performance but use extra storage.
- Too many indexes can slow down writes.
- Use indexes on frequently queried columns.

EXAMPLE

CREATE INDEX idx_email
ON users(email);

3 READ REPLICAS



- Create read-only copies of your database.
- Offload read traffic from the primary database.
- Improve performance and availability.
- Common in RDS and Aurora.

USE CASES

- High read traffic applications
- Reporting and analytics

4 PARTITIONING



- Split large datasets into smaller parts (partitions).
- Improves performance and manageability.
- Can be done by range, list, or hash.
- Used in both relational databases and NoSQL databases.

EXAMPLE

Partition data by date (e.g., monthly orders).

5 SHARDING CONCEPTS



- Split data across multiple databases (shards).
- Each shard holds a subset of the total data.
- Helps scale horizontally and handle large datasets.
- Requires a sharding strategy (e.g., by user_id, region).

EXAMPLE

User data shard 1, shard 2, shard 3
based on user_id range.

6 CONNECTION MANAGEMENT



- Manage database connections efficiently.
- Too many connections can overload the database.
- Use connection pooling.
- Close idle connections.
- Important for high-traffic applications.

BEST PRACTICES

- Use a connection pool (e.g., HikariCP)
- Set reasonable connection limits

7 RDS PROXY



Amazon
RDS Proxy

- Manages database connections for you.
- Improves application scalability.
- Reduces connection overhead and failover time.
- Works with RDS and Aurora.
- Automatically handles connection pooling.

USE CASES

- Applications with many short-lived connections
- Serverless and containerized applications

8 DYNAMODB PARTITION KEYS



- Determine how data is distributed across partitions.
- A good partition key ensures even data distribution.
- Choose keys with high cardinality (many unique values).
- Avoid keys that cause uneven traffic.

EXAMPLE

user_id, order_id, or a combination
like user_id#order_id.

9 HOT PARTITIONS



- Happen when a partition key receives too much traffic.
- Can cause throttling and poor performance.
- Avoid low-cardinality keys (e.g., status = "active").
- Use a more distributed key or add randomness (e.g., a suffix).

SOLUTION

Use high-cardinality keys
and monitor traffic patterns.

10 DATA GROWTH



- Plan for increasing data over time.
- Consider storage limits, performance, and cost.
- Use partitioning, sharding, and lifecycle policies.
- Archive or delete old data when appropriate.
- Monitor growth and set up alerts.

11 DATABASE BOTTLENECKS



Common bottlenecks include:

- High read or write traffic
- Long-running queries
- Insufficient indexes
- Connection limits
- Large unoptimized queries
- Poorly designed schema

HOW TO FIX

- ✓ Identify with monitoring
- ✓ Optimize queries
- ✓ Add indexes
- ✓ Scale (read replicas, sharding)
- ✓ Use caching (e.g., ElastiCache)
- ✓ Review and improve schema



KEY TAKEAWAY

A scalable data layer is designed with the right keys, indexes, distribution, and operational safeguards from the start. Understand your access patterns, plan for growth, and eliminate bottlenecks before they become problems.

CACHING FOR PERFORMANCE AND SCALE

Store data closer, reduce latency, handle more traffic, and improve overall system performance.

1 WHY CACHING EXISTS



- Reduces latency by serving data faster.
- Decreases load on databases and backend services.
- Handles traffic spikes and improves scalability.
- Makes applications more resilient and cost efficient.

Cache stores a copy of frequently accessed data so it can be served quickly without querying the original source.

2 CLOUDFRONT CACHING



- Edge caching for static and dynamic content.
- Stores content in global edge locations.
- Reduces latency for users worldwide.
- Supports cache behaviors, TTL, and invalidation.
- Ideal for websites, APIs, images, videos, and files.

Use CloudFront to cache content closer to your users.

3 ELASTICACHE



- Managed in-memory caching service.
- Supports Redis and Memcached.
- Ultra-low latency (microseconds).
- Improves application performance.
- Commonly used for sessions, user data, and frequently accessed data.

Use ElastiCache when you need fast, in-memory caching.

4 REDIS



- Popular open-source in-memory data store.
- Supports rich data structures (strings, hashes, lists, sets, etc.).
- Used for caching, sessions, rate limiting, leaderboards, and real-time features.
- Available via ElastiCache or self-managed.

Redis is the most common caching engine in AWS.

5 APPLICATION CACHING



- Cache data inside your application (in-memory).
- Reduces repeated calls to external services.
- Good for simple and short-lived data.
- Examples: in-memory cache using Redis client, local cache.

Use application caching for data that changes infrequently and is safe to store in memory.

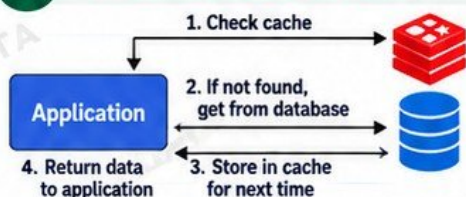
6 DATABASE CACHING



- Use a cache layer (e.g., Redis/ElastiCache) in front of your database.
- Reduces read load and improves response time.
- Common for read-heavy workloads.
- Still keep the database as the source of truth.

Cache frequently read data, but always keep the database as the source of truth.

7 CACHE-ASIDE PATTERN



- Application checks cache first.
- If not found, it queries the database.
- Stores the result in cache for future requests.
- Common and widely used pattern.

8 TTL (TIME TO LIVE)



- Defines how long data stays in the cache.
- Helps keep data fresh.
- Prevents stale data.
- Choose an appropriate TTL based on how often data changes.

Short TTL = fresher data
Long TTL = better performance

9 CACHE INVALIDATION



- Remove or update cached data when the underlying data changes.
- Can be time-based (TTL) or event-driven.
- Important to prevent users from seeing stale data.
- Techniques: TTL, manual invalidation, event triggers (e.g., after a database update).

10 CACHE MISSES



- Happen when requested data is not in the cache.
- Causes a fallback to the original data source (e.g., database).
- Higher latency compared to cache hits.
- Can impact performance if it happens frequently.
- Monitor cache miss rate and optimize.

11 CACHE STAMPEDE



- Occurs when many requests try to fetch the same data at the same time after it expires.
- Can overload the database.
- Mitigation techniques:
 - Use TTL with jitter
 - Early refresh / refresh ahead
 - Request coalescing (locking)

12 INTERVIEW QUESTION

WHERE SHOULD WE CACHE?



- Static content → CloudFront
- User sessions → ElastiCache (Redis)
- Database query results → ElastiCache
- Application data → Application cache
- Expensive or slow API responses → ElastiCache
- Frequently accessed, rarely changing data → Any appropriate cache layer
- Always explain your reasoning based on: access patterns, data freshness, latency, and scale.

Events
Drive
Scalable
Systems

QUEUES, EVENTS, AND ASYNCHRONOUS ARCHITECTURE

Build scalable, decoupled, and resilient systems using messaging and event-driven patterns.

Asynchronous
Systems
Stronger
Together

1 SQS (SIMPLE QUEUE SERVICE)



- Fully managed message queue.
- Decouples producers and consumers.
- Supports standard and FIFO queues.
- Handles variable workloads.
- Highly scalable and durable.
- Used for asynchronous processing.

USE CASES

- Background jobs
- Order processing
- Buffering traffic spikes
- Microservices

2 SNS (SIMPLE NOTIFICATION SERVICE)



- Fully managed pub/sub service.
- Sends messages to multiple subscribers.
- Supports many protocols (SQS, Lambda, email, HTTP).
- Useful for fan-out architecture.
- Often used with SQS.

USE CASES

- Event notifications
- Application alerts
- Fan-out to multiple services
- Real-time messaging

3 EVENTBRIDGE



- Serverless event bus.
- Connects apps using events.
- Built-in integrations with AWS services and SaaS apps.
- Supports event filtering, routing, and transformation.
- Used for event-driven architectures at scale.

USE CASES

- Service-to-service events
- Scheduled events
- Third-party integrations
- Build event-driven systems

4 KINESIS



- Managed service for real-time data streaming.
- Handles high-throughput event data.
- Supports Kinesis Data Streams, Data Firehose, and Data Analytics.
- Used for real-time analytics and processing.

USE CASES

- Real-time log processing
- IoT and sensor data
- Clickstream data
- Real-time analytics

5 STEP FUNCTIONS



- Managed workflow orchestration.
- Coordinates multiple AWS services.
- Handles complex, long-running processes.
- Built-in error handling, retries, and state management.
- Visual workflow designer.

USE CASES

- Order processing workflows
- Data pipelines
- Multi-step business logic
- Error handling and retries

6 PRODUCERS AND CONSUMERS



- Producers send messages to a queue or event bus.
- Consumers process the messages.
- They can scale independently.
- Decouples components and improves resilience.
- Multiple consumers can process messages in parallel.

7 DECOUPLING SERVICES



- Services communicate through queues or events, not direct calls.
- Reduces tight coupling.
- Improves scalability, reliability, and maintainability.
- Allows independent deployment and scaling.

8 BUFFERING TRAFFIC SPIKES



- Queues absorb sudden increases in traffic.
- Prevents system overload.
- Consumers can process at a steady pace.
- Helps maintain stability during peak loads.

9 RETRIES



- Automatically retry failed messages.
- Use exponential backoff.
- Avoid immediate, repeated retries.
- Combine with Dead-Letter Queues for persistent failures.
- Helps handle temporary issues like network errors or throttling.

10 DEAD-LETTER QUEUES (DLQ)



- Stores messages that cannot be processed successfully.
- Helps isolate and troubleshoot problematic messages.
- Prevents bad messages from blocking the queue.
- Set a maximum receive count before moving to DLQ.

11 IDEMPOTENCY



- Ensures the same message can be processed multiple times without changing the result.
- Important because messages may be delivered more than once.
- Use unique IDs, conditional writes, or checks to prevent duplicates.
- Essential for reliable, fault-tolerant systems.

12 AT-LEAST-ONCE DELIVERY



- Most AWS messaging services (like SQS) provide at-least-once delivery.
- A message might be delivered more than once.
- Design your consumers to be idempotent.
- Handle duplicate messages safely.

13 SYNCHRONOUS vs ASYNCHRONOUS DESIGN

Synchronous (Request/Response)

- Client waits for a response.
- Lower latency for simple operations.
- Tightly coupled.
- Not ideal for long-running tasks.
- Examples: API Gateway → Lambda, ALB → EC2.

Asynchronous (Queues/Events)

- Client does not wait for a response.
- Better for long-running or high-volume tasks.
- Decoupled and more resilient.
- Handles traffic spikes.
- Examples: SQS, SNS, EventBridge, Kinesis, Step Functions.



KEY TAKEAWAYS

- Use queues and events to build scalable and resilient systems.
- Choose the right service for your use case.
- Design for failures, retries, and idempotency.
- Decouple services and handle traffic spikes.
- Understand the difference between synchronous and asynchronous architectures.
- Always consider monitoring and DLQs.

"Asynchronous systems don't just handle more traffic, they give you a more resilient future."

Same
Better
Stronger

@veriqta

veriqta

veriqta

veriqta

@veriqta

@veriqta

Learn
Build
Grow

VERIQTA

TECH KNOWLEDGE FOR A BRIGHTER TOMORROW

DESIGNING FOR HIGH AVAILABILITY

Build systems that stay available, even when things fail.

1 AVAILABILITY vs DURABILITY



Availability

- System is accessible when needed
- Measured in uptime (e.g. 99.9%)



Durability

- Data is not lost
- Persists even after failures
- Measured in durability (e.g. 11 9s for S3)

2 SINGLE POINTS OF FAILURE



- A single point of failure (SPoF) is any component that can bring down the entire system.

Examples:

- One server
- One database
- One Availability Zone
- One network link

3 MULTI-AZ ARCHITECTURE



- Deploy resources across multiple Availability Zones (AZs)
- Each AZ is isolated with independent power, networking, and hardware
- Improves availability and resilience
- Used for compute, databases, and load balancing

4 LOAD BALANCING



- Distributes traffic across healthy instances
- Improves availability and scalability
- Common options: ALB, NLB
- Performs health checks
- Routes traffic away from unhealthy targets

5 AUTO SCALING



- Automatically adjusts the number of instances based on demand
- Replaces unhealthy instances
- Helps maintain availability during traffic spikes or failures

6 DATABASE FAILOVER



- Uses Multi-AZ deployments
- Automatic failover to standby database
- Minimizes downtime
- Examples: RDS Multi-AZ, Aurora (automatic failover)
- Applications should be designed to handle failovers

7 REPLICATION



- Keeps copies of data across AZs or Regions
- Synchronous or asynchronous replication
- Used for databases (RDS, Aurora, DynamoDB)
- Ensures data is available even if one AZ fails

8 HEALTH CHECKS



- Continuously checks the health of instances, services, and endpoints
- Unhealthy resources are removed from traffic
- Used by load balancers, Auto Scaling, and Route 53

9 REDUNDANCY



- Multiple copies of critical components
- Reduces the impact of failures
- Includes compute, storage, networking, and databases
- No single component should be critical for the entire system

10 GRACEFUL DEGRADATION



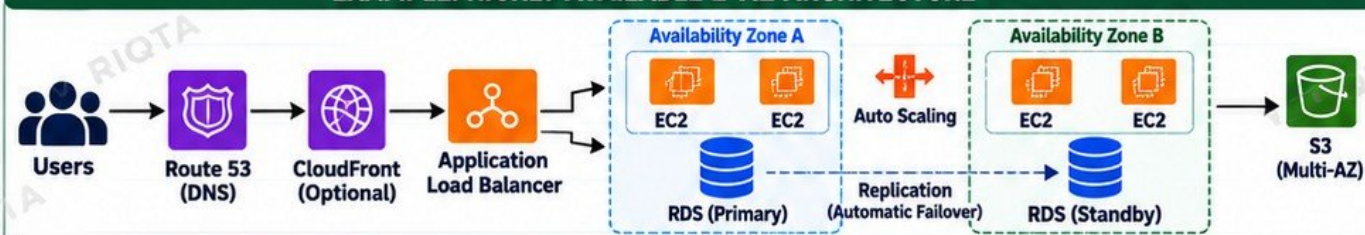
- System continues to work with reduced functionality
- Non-critical features may be disabled during failures
- Provides a better user experience than a complete outage
- Example: Show cached data if database is unavailable

11 WHAT HAPPENS WHEN ONE AVAILABILITY ZONE FAILS?



- Load balancer stops sending traffic to unhealthy instances in the failed AZ
- Auto Scaling launches new instances in a healthy AZ
- Multi-AZ databases automatically fail over to a healthy AZ
- DNS (Route 53) can route traffic to healthy endpoints
- The system continues to operate with minimal or no downtime
- Users may experience little to no impact if designed correctly

EXAMPLE: HIGHLY AVAILABLE 2-AZ ARCHITECTURE



KEY TAKEAWAY

Design for failure, not for perfection. Assume things will fail and build systems that can recover automatically.

Plan
Recover
Stay Ready

DISASTER RECOVERY AND MULTI-REGION DESIGN

Design for failure. Keep your business running.

1 RECOVERY TIME OBJECTIVE (RTO)



- The maximum acceptable time to restore the service after a failure.
- Defines how quickly your application must be back online.
- Shorter RTO usually means higher cost.

Example: RTO of 1 hour means the service must be restored within 1 hour of a major failure.

2 RECOVERY POINT OBJECTIVE (RPO)



- The maximum acceptable amount of data loss measured in time.
- Defines how much data you can afford to lose during a failure.
- Shorter RPO usually means more frequent backups/replication and higher cost.

Example: RPO of 15 minutes means you can lose at most 15 minutes of data.

3 BACKUP AND RESTORE



- Regular backups of data and configurations.
- Stored in durable storage (e.g. S3, AWS Backup).
- Test your backups regularly.
- Define retention policies.
- Know the restore process and time.

Backups are essential, but restore time must also meet your RTO.

4 PILOT LIGHT



- Minimal infrastructure in a secondary region.
- Core services are running (but limited capacity).
- Scale up during a disaster.
- Lower cost, moderate recovery time.

Good for: applications that can tolerate some downtime.

5 WARM STANDBY



- A scaled-down but fully functional environment in a secondary region.
- Can be scaled up quickly when needed.
- Faster recovery than pilot light.
- Higher cost than pilot light.

Good for: important applications with moderate RTO.

6 ACTIVE-PASSIVE



- Primary region handles all traffic.
- Secondary region is on standby.
- Failover happens during a disaster.
- Lower cost.

Good for: many business applications with clear primary region.

7 ACTIVE-ACTIVE



- Both regions are active and serve traffic.
- Automatic failover.
- Highest availability and resilience.
- Higher cost and more complexity.

Good for: global applications with high availability requirements.

8 CROSS-REGION REPLICATION



- Replicate data across AWS regions.
- Used by services like RDS (read replicas), Aurora Global Database, DynamoDB global tables, S3 Cross-Region Replication.
- Helps meet low RPO.

Keeps your data safe, even if an entire region is lost.

9 ROUTE 53 FAILOVER



- Automatically routes traffic to a healthy region when the primary region is unavailable.
- Uses health checks.
- Supports failover, latency, weighted, and geolocation routing.

Helps users reach your application during regional failures.

10 COST vs RECOVERY CAPABILITY



- Shorter RTO and RPO increase cost.
- Multi-region solutions are more expensive than single-region.
- Balance business risk, user impact, and budget.
- Not every application needs active-active.

Choose the right DR strategy for your business needs.

11 WHEN MULTI-REGION IS ACTUALLY JUSTIFIED



- Your application is business critical (e.g. finance, healthcare, e-commerce).
- You have global users and need low latency.
- You must meet strict regulatory or compliance requirements.
- A regional outage would cause significant revenue loss.
- You need high availability (e.g. 99.99% or higher).
- The cost is justified by the business impact.

Use multi-region when the value of higher availability outweighs the additional cost and complexity.



KEY TAKEAWAY

Disaster recovery is not a product, it is a strategy. Understand your risk, define your RTO and RPO, and choose the right architecture.

Learn
Build
Grow

@veriqta

veriqta

veriqta

veriqta

@veriqta

@veriqta

VERIQTA

TECH KNOWLEDGE FOR A BRIGHTER TOMORROW

Better
Engineers
Brighter
Careers

SECURITY ARCHITECTURE IN SYSTEM DESIGN

Security Builds Trust

Design secure systems. Protect your data, applications, and users.

1 SHARED RESPONSIBILITY MODEL

Your Responsibility (Customer)

- Data and application security
- Identity and access management
- Network and firewall configuration
- Operating system and patching
- Security of your workloads



AWS Responsibility

- Physical infrastructure
- Compute, storage, networking
- Hardware and facilities
- Global infrastructure security

Security is a shared responsibility. Know what is yours.

2 IAM, ROLES AND LEAST PRIVILEGE



IAM

- Manages access to AWS resources
- Users, groups, policies



Roles

- Temporary credentials
- Used by services, applications, and users
- More secure than long-term access keys



Least Privilege

- Grant only the minimum permissions required
- Reduces security risks
- Follow the principle of least privilege

Give the right access, not full access.

3 SECURITY GROUPS



- Acts as a virtual firewall
- Controls inbound and outbound traffic
- Stateful (allows return traffic)
- Applied to EC2, RDS, ALB, etc.
- Use least privilege rules
- Allow only required ports and IP ranges

Security Groups protect your resources at the network level.

4 AWS WAF



- Protects your applications from common web threats
- Filters malicious traffic (e.g. SQL injection, XSS)
- Works with CloudFront, ALB and API Gateway
- Use managed rule groups

Use AWS WAF to protect your applications at the edge.

5 AWS SHIELD



- Helps protect against DDoS attacks
- Shield Standard is free (for all AWS customers)
- Shield Advanced provides enhanced protection and DDoS response team
- Integrates with Route 53, CloudFront and other services

Use AWS Shield to keep your applications available.

6 KMS (KEY MANAGEMENT SERVICE)



- Creates and manages encryption keys
- Encrypts data at rest
- Used with S3, RDS, EBS and other AWS services
- Supports key rotation
- Helps you meet compliance requirements

Use KMS to protect and manage your encryption keys.

7 SECRETS MANAGER



- Stores and manages secrets (e.g. database passwords, API keys)
- Automatically rotates secrets
- Integrates with AWS services
- More secure than hardcoding credentials

Use Secrets Manager to keep your secrets safe.

8 ENCRYPTION AT REST



- Protects data stored on disk or in storage (e.g. S3, EBS, RDS)
- Use KMS for key management
- Enable encryption by default
- Helps prevent data breaches

Always encrypt data at rest.

9 ENCRYPTION IN TRANSIT



- Protects data while it is moving
- Use TLS/SSL (HTTPS)
- Encrypt traffic between clients and servers
- Encrypt traffic between AWS services
- Prevents eavesdropping and tampering

Always use encryption in transit (e.g. HTTPS).

10 PRIVATE RESOURCES



- Keep sensitive resources in private subnets
- Use NAT Gateway for outbound access
- Restrict public access
- Use VPC endpoints when possible
- Only expose what needs to be public

Keep your critical resources private.

11 SECURITY BOUNDARIES



- Use VPCs to isolate resources
- Create separate accounts for different environments
- Use AWS Organizations and SCPs
- Implement network segmentation
- Define clear access boundaries

Set clear boundaries to reduce risk and contain potential damage.

12 NEVER DESIGNING SECURITY AS AN AFTERTHOUGHT



- Security must be part of the design from day one
- Consider security in every architectural decision
- Think about threats, not just functionality
- Build with compliance and governance in mind
- Review and iterate as requirements evolve

Security is not a feature. It is a foundation.



KEY TAKEAWAY

A secure architecture protects your users, your data, and your business. Design with security from the beginning.

OBSERVABILITY AND PRODUCTION OPERATIONS

See what's happening. Find problems early. Keep systems healthy.

Monitor
Detect
Respond
Keep Running

1 METRICS



- Numerical data about system performance
- Collected over time
- Examples: CPU, memory, latency, request count
- Helps you understand trends and detect anomalies

Metrics show you how your system is behaving.

2 LOGS



- Detailed records of events
- Useful for troubleshooting
- Examples: application logs, system logs, error logs
- Stored in CloudWatch Logs or other tools
- Helps you find the root cause of issues

Logs tell you what happened.

3 TRACES



- Track a request as it travels through multiple services
- Shows the full request flow
- Helps identify where latency occurs
- Use AWS X-Ray for distributed tracing

Traces show you the journey of a request.

4 CLOUDWATCH



- Central monitoring service in AWS
- Collects metrics, logs and alarms
- Create dashboards
- Set alarms and notifications
- Monitor AWS resources and applications

CloudWatch gives you visibility into your infrastructure and applications.

5 CLOUDTRAIL



- Records API activity in your AWS account
- Shows who did what, when and from where
- Useful for security, compliance and auditing
- Helps detect unauthorized changes

CloudTrail lets you track and audit activity in your environment.

6 AWS X-RAY



- Distributed tracing service
- Visualizes request flow across services
- Helps find performance bottlenecks
- Integrates with CloudWatch
- Useful for microservices and serverless applications

X-Ray helps you see inside your application.

7 DASHBOARDS



- Visualize key metrics and logs
- Track system health in real time
- Use CloudWatch dashboards or tools like Grafana
- Common dashboards: Infrastructure, application, security, business metrics

Dashboards give you a real-time view of your system.

8 ALERTS



- Notify you when something is wrong
- Set thresholds for metrics (e.g. high CPU, error rate)
- Use CloudWatch Alarms
- Send notifications via SNS, email, or Slack
- Helps you respond quickly

Alerts help you detect and respond to problems early.

9 SERVICE HEALTH



- Check the health of AWS services using the AWS Health Dashboard
- Monitor the health of your own services (e.g. EC2, RDS, ECS, Lambda)
- Use health checks and status endpoints

Service health tells you if your infrastructure is running as expected.

10 APPLICATION HEALTH



- Monitor application performance and errors
- Track key business metrics
- Use logs, metrics and traces together
- Set up synthetic tests (e.g. API health checks)
- Monitor user experience (e.g. response time, error rate)

Application health ensures your users have a good experience.

11 SLI, SLO, AND SLA BASICS



- SLI (Service Level Indicator): A measurable metric (e.g. request success rate)
- SLO (Service Level Objective): Target level for the SLI (e.g. 99.9% success rate)
- SLA (Service Level Agreement): Formal commitment, often with penalties

SLIs measure. SLOs set the target. SLAs are the commitment.

12 HOW WOULD YOU KNOW THE ARCHITECTURE IS FAILING?



- High error rates (4xx, 5xx)
- Increased latency
- Services are unhealthy
- Alarms are triggered
- Dashboards show abnormal patterns
- Users report issues
- Dependencies are down (e.g. database, external API)
- System is not meeting SLOs

A failing architecture shows signs in metrics, logs, traces, and user impact.



KEY TAKEAWAY

Observability is not optional. You cannot fix what you cannot see. Monitor, set alerts, and continuously review your system's health.

Learn
Build
Grow



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

VERIQTA

TECH KNOWLEDGE FOR A BRIGHTER TOMORROW

Better
Engineers
Brighter
Careers

RELIABILITY PATTERNS AND FAILURE HANDLING

Design for failure. Build systems that recover.

Build
Resilient
Systems

1 TIMEOUTS



- Set a maximum time to wait for a response
- Prevents requests from hanging indefinitely
- Use reasonable timeout values based on the service (e.g. 1-5 seconds)
- Different timeouts for different dependencies

If a service is slow, fail fast and try an alternative.

2 RETRIES



- Automatically retry failed requests
- Useful for temporary failures (e.g. network issues, 5xx errors)
- Do not retry non-retriable errors (e.g. 4xx)
- Always combine with exponential backoff

Retries can turn a temporary failure into a success.

3 EXPONENTIAL BACKOFF



- Increase the wait time between retries
- Example: 1s → 2s → 4s → 8s → 16s
- Reduces pressure on a struggling service
- Helps prevent retry storms

Give the system time to recover.

4 JITTER



- Adds random variation to the backoff time
- Prevents many clients from retrying at the same time
- Reduces the chance of retry storms
- Common jitter types: Full, Equal, Decorrelated

Jitter spreads out retries and keeps the system stable.

5 CIRCUIT BREAKERS



- Stops calling a failing service after a threshold of failures
- Moves to an open state
- After a cooldown period, it tries again (half-open)
- Prevents cascading failures

Fail fast, recover gracefully, keep your system stable.

6 IDEMPOTENCY



- Ensure the same request can be safely retried multiple times
- Prevents duplicate operations (e.g. double payments, duplicate orders)
- Use idempotency keys where supported

A retry should not create unwanted side effects.

7 DEAD-LETTER QUEUES



- Store messages that cannot be processed after multiple retries
- Helps with debugging and manual processing
- Common with SQS, Kafka and other messaging systems
- Set a retry limit before sending to the DLQ

Don't lose failed messages. Send them to a DLQ.

8 RATE LIMITING



- Limits the number of requests a client can make
- Prevents abuse and protects your services
- Can be based on IP, user, or API key
- Helps keep the system available for everyone

Control traffic to keep your system healthy.

9 BACKPRESSURE



- Signals to upstream services that the system is overloaded
- Slows down the rate of incoming requests
- Prevents resource exhaustion
- Common in streaming and queue-based systems

It's better to slow down than to crash.

10 GRACEFUL DEGRADATION



- Continue to provide core functionality during partial failures
- Disable non-critical features
- Return cached data when possible
- Show a friendly message instead of an error page

Keep the essential features running, even when parts fail.

11 DEPENDENCY FAILURES



- External services can and will fail
- Design for these failures with timeouts, retries, circuit breakers, and fallbacks
- Use multiple providers where appropriate
- Monitor dependencies continuously

Your system is only as reliable as its dependencies.

12 RETRY STORMS



- Happens when many clients retry at the same time
- Can make a bad situation worse
- Use exponential backoff and jitter
- Set reasonable retry limits
- Combine with circuit breakers and rate limiting

Retries must be controlled to avoid making outages worse.

13 DESIGNING FOR FAILURE INSTEAD OF ASSUMING SUCCESS



- Failures are normal in distributed systems
- Plan for network issues, service outages, and slow responses
- Use resilience patterns (timeouts, retries, circuit breakers, etc.)
- Design for observability so you can detect and respond quickly

- Test failure scenarios (e.g. chaos engineering)
- Have runbooks and incident response plans
- Assume things will fail and build systems that recover automatically

A resilient system is not one that never fails, but one that recovers quickly.

Learn
Build
Grow



VERIQTA

TECH KNOWLEDGE FOR A BRIGHTER TOMORROW

Better
Engineers
Brighter
Careers

COST-AWARE ARCHITECTURE

Design smart. Deliver value. Optimize for cost and performance.

Think
Cost
Build
Sustainable

1 COST AS A SYSTEM-DESIGN REQUIREMENT



- Cost is a non-functional requirement.
- Understand budget constraints.
- Design for cost efficiency, not just functionality.
- Consider total cost of ownership (TCO).
- Make cost a key part of trade-offs and decisions.

A good design meets requirements at the right cost.

2 ON-DEMAND vs RESERVED vs SPOT



On-Demand

- Pay per use
- No commitment
- Higher cost
- Good for unpredictable workloads



Reserved

- 1 or 3 years
- Lower cost (up to 72%)
- Requires commitment
- Good for stable workloads



Spot

- Very low cost (up to 90%)
- Can be interrupted
- Good for fault-tolerant workloads (batch, CI/CD, stateless)

Choose the right pricing model based on workload stability and risk tolerance.

3 SERVERLESS ECONOMICS



- Pay only for what you use.
- No server management cost.
- Scales automatically.
- Ideal for variable or unpredictable workloads.
- Consider cold start impact and execution time.
- Compare with containers/EC2 for long-running workloads.

Serverless can be cost-effective but is not always the cheapest option.

4 DATA TRANSFER COSTS



- Data transfer between AWS services can be costly.
- Cross-Region data transfer is expensive.
- Internet data transfer (egress) incurs charges.
- Use CloudFront, caching, and Regional architectures to reduce costs.

Minimize unnecessary data transfer to keep costs under control.

5 NAT GATEWAY CONSIDERATIONS



- NAT Gateways are not free.
- Charged per hour and per GB of data processed.
- Can become a significant cost in large environments.
- Consider alternatives (VPC endpoints, NAT instances, to reduce architectures).

Design your network to avoid unnecessary NAT Gateway costs.

6 STORAGE TIERS



- Use the right storage class for the right data.
- S3 Standard: frequently accessed data.
- S3 Intelligent-Tiering: automatically moves data.
- S3 Standard-IA / One Zone-IA: infrequent access.
- S3 Glacier / Deep Archive: long-term storage.

Store data based on access patterns, not just convenience.

7 CLOUDFRONT AND CACHING



- CloudFront reduces latency and data transfer costs.
- Caches content at edge locations.
- Offloads traffic from your origin (EC2, S3, ALB).
- Use appropriate cache policies and TTL.
- Great for static and dynamic content.

Caching improves performance and reduces costs.

8 OVERPROVISIONING



- Provisioning more resources than needed wastes money.
- Common in compute, storage, and databases.
- Right-size based on real usage metrics.
- Use Auto Scaling instead of fixed large instances.
- Review and remove idle resources.

Avoid overprovisioning. Pay for what you use.

9 RIGHT-SIZING



- Choose the correct instance size and type.
- Use monitoring data (CloudWatch) to right-size.
- Upgrade or downgrade based on actual usage.
- Consider ARM-based instances (Graviton) for better price-performance.

Right-sizing gives you the best performance at the lowest cost.

10 COST vs PERFORMANCE



- Higher performance usually means higher cost.
- Find the right balance for your use case.
- Use managed services to reduce operational overhead.
- Consider caching, async processing, and batching to improve efficiency.

Don't overpay for performance you don't need.

11 COST vs AVAILABILITY



- Higher availability (multi-AZ, multi-Region) increases cost.
- Decide the right availability level based on business requirements.
- Not every application needs multi-Region.
- Use cost-effective DR strategies (e.g. pilot light) where possible.

Match availability to business needs and budget.

12 EXPLAINING FINANCIAL TRADE-OFFS



- Clearly communicate your cost assumptions.
- Explain why you chose a specific architecture.
- Discuss alternatives and their cost impact.
- Show that you understand the business value, not just the technology.
- Be ready to adjust the design if budget constraints change.

In an interview, show that you can design for both technical and business success.



KEY TAKEAWAY

Good architecture is not the most expensive. It is the right solution that delivers the required value at an optimal cost.

Learn
Build
Grow



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

VERIQTA

TECH KNOWLEDGE FOR A BRIGHTER TOMORROW

Better
Engineers
Brighter
Careers

DESIGN INTERVIEW, HIGHLY AVAILABLE WEB APPLICATION

Design
Build
Scale
Succeed

Design a scalable, secure, and highly available web application on AWS.

1 REQUIREMENTS GATHERING



- Understand the problem and goals
- Ask clarifying questions
- Identify functional requirements (e.g. user registration, login, content)
- Identify non-functional requirements (e.g. high availability, performance)
- Estimate users, traffic, and data
- Define constraints (cost, compliance, etc.)

2 ROUTE 53 (DNS)



- Provides reliable and low-latency DNS
- Routes users to the application
- Supports health checks and failover
- Can use latency or geolocation routing

3 CLOUDFRONT (CDN)



- Caches static content globally
- Reduces latency
- Protects origin servers
- Works with S3 and ALB
- Improves performance and scalability

4 APPLICATION LOAD BALANCER



- Distributes traffic across healthy instances
- Works across multiple AZs
- Performs health checks
- Routes traffic to target groups
- Supports HTTPS and SSL termination

5 MULTI-AZ APPLICATION TIER



- Deploy application instances in multiple AZs
- Use Auto Scaling for high availability
- Keep instances stateless
- Store sessions in a shared store (e.g. ElastiCache)
- Automatically replace unhealthy instances

6 AUTO SCALING



- Automatically adjusts capacity
- Handles traffic spikes
- Maintains availability and performance
- Store across multiple AZs
- Uses CloudWatch metrics for scaling

11 MONITORING



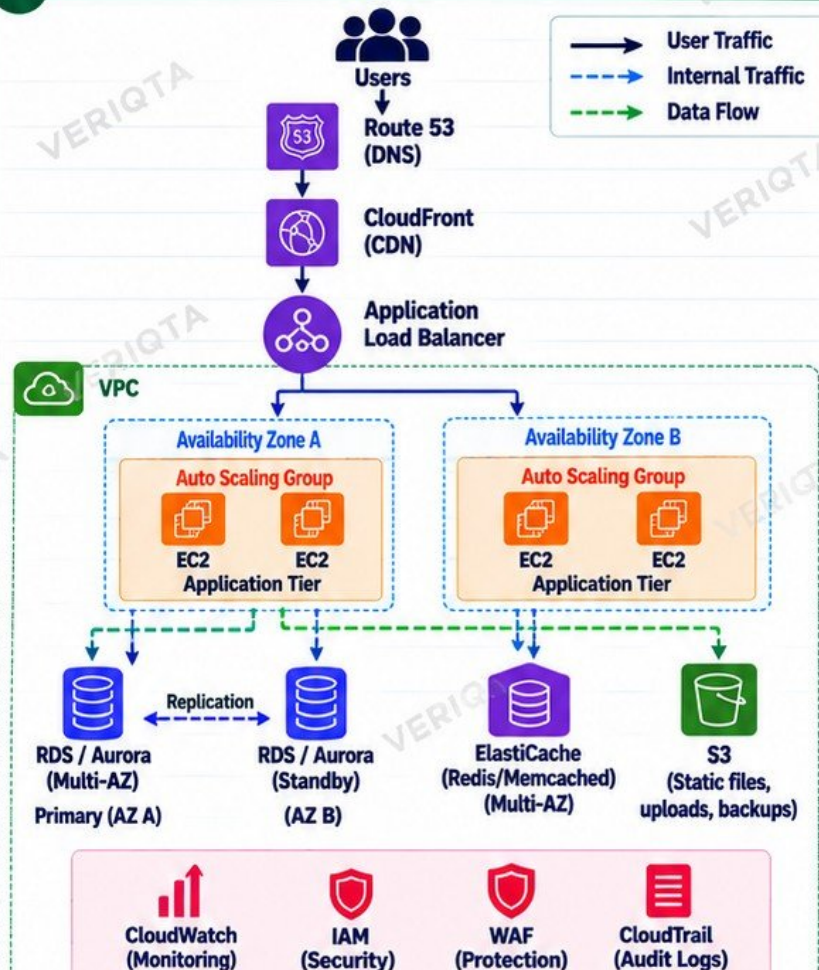
- Use CloudWatch for metrics, logs, and alarms
- Monitor ALB, EC2, RDS, ElastiCache, etc.
- Set up dashboards and alerts
- Use CloudTrail for audit and compliance

12 SECURITY



- Use IAM with least privileges
- Implement Security Groups and NACLs
- Use AWS WAF for web protection
- Encrypt data at rest and in transit
- Follow security best practices

7 HIGH-LEVEL ARCHITECTURE DIAGRAM



8 RDS / AURORA



- Managed relational database
- Multi-AZ for high availability
- Automatic failover
- Read replicas for scaling
- reads
- Backups and point-in-time recovery

9 ELASTICACHE



- In-memory cache (Redis/Memcached)
- Reduces database load
- Improves application performance
- Multi-AZ for high availability
- Used for sessions, caching, etc.

10 S3



- Stores static content (images, videos, etc.)
- Highly durable and scalable
- Works with CloudFront
- Used for uploads, backups, logs, etc.

13 FAILURE ANALYSIS



- What happens if an AZ fails?
- How does the system detect failures?
- How is traffic rerouted?
- What components are redundant?
- How are data and sessions protected?

14 SCALING DISCUSSION



- How to handle 10x traffic?
- What to scale (compute, database, cache)?
- Use Auto Scaling and read replicas
- Leverage caching and CDN
- Consider cost and performance trade-offs

15 ARCHITECTURE WALKTHROUGH



- Explain the full design end-to-end
- Justify each AWS service choice
- Discuss trade-offs and alternatives
- Address reliability, security, and cost
- Show how the design meets requirements

★ KEY TAKEAWAY

A well-designed system is resilient, scalable, secure, and cost-effective. Always think about failure, failure, scale, and trade-offs.

DESIGN INTERVIEW, LARGE-SCALE FILE AND MEDIA PLATFORM

Plan
Build
Scale
Deliver

Design a scalable, secure, and highly available platform for uploading, processing, and delivering files and media.

1 UPLOAD REQUIREMENTS



- What types of files (images, videos, documents, etc.)?
- Expected file sizes (KB, MB, GB)?
- Number of users and upload rate?
- Geographic distribution of users?
- Latency and availability requirements?
- Any processing (e.g. video transcoding, image resizing)?
- Retention period and lifecycle rules?

2 AMAZON S3



- Stores uploaded files
- Highly durable (11 9s)
- Scales automatically
- Supports large files (up to 5 TB)
- Use lifecycle policies for cost optimization
- Versioning for data protection

3 PRESIGNED URLS



- Allow secure, temporary access to S3
- Enable direct browser/app uploads
- No need to expose AWS credentials
- Set expiration time
- Use IAM policies to restrict access

4 CLOUDFRONT



- Global content delivery network (CDN)
- Delivers files with low latency
- Caches files at edge locations
- Works with S3 (origin)
- Supports signed URLs for secure content
- Improves user experience globally

5 METADATA DATABASE



- Stores file metadata (file name, size, type, owner, status)
- Use Amazon RDS (relational) or DynamoDB (NoSQL)
- Design for high availability and scalability
- Index frequently queried fields
- Keep metadata and file storage separate

6 AMAZON SQS



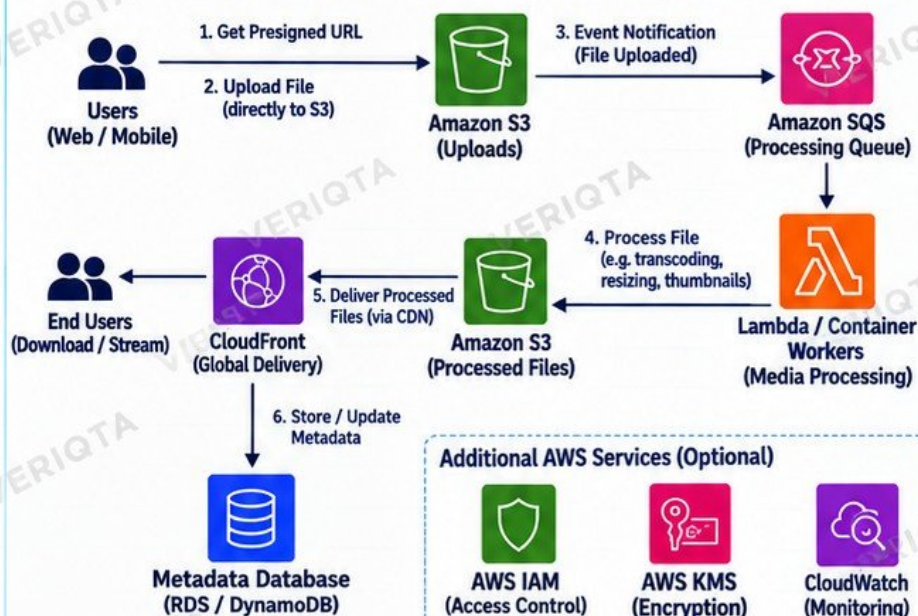
- Decouples upload process from processing
- Buffers messages during traffic spikes
- Ensures reliable, asynchronous processing
- Use dead-letter queues (DLQ) for failed messages
- Scales automatically

7 LAMBDA / CONTAINER WORKERS



- Consume messages from SQS
- Run media processing tasks
- Use AWS Lambda for smaller tasks
- Use ECS/EKS for long-running or heavy processing
- Auto scales based on queue size
- Handle failures and retries

8 HIGH-LEVEL ARCHITECTURE DIAGRAM



9 MEDIA PROCESSING



- Transcode videos to multiple formats (e.g. HLS, MP4)
- Generate thumbnails
- Extract metadata (duration, resolution, etc.)
- Use AWS Elemental MediaConvert or custom FFmpeg workers
- Store processed files in S3
- Update metadata database

10 ASYNCHRONOUS WORKFLOWS



- Use event-driven architecture
- Decouple upload, processing, and notification
- Use SQS, EventBridge, or SNS
- Handle retries and failures
- Send notifications when processing is complete
- Build scalable, loosely coupled components

11 LARGE-FILE HANDLING



- Use multipart uploads (for files > 100 MB)
- Supports files up to 5 TB
- Improves reliability and resumability
- Use SDKs or presigned URLs with multipart upload
- Handle network failures
- Show progress to users

12 SECURITY



- Use IAM with least privilege
- Presigned URLs with short expiration
- Encrypt files at rest (S3 + KMS)
- Encrypt data in transit (HTTPS)
- Validate file types and size
- Use AWS WAF for API protection
- Scan files for malware (e.g. with Amazon GuardDuty or third-party)

13 GLOBAL DELIVERY



- Use CloudFront for global content delivery
- Store files in S3 with proper storage class
- Use multiple edge locations
- Consider multi-region setup for critical workloads
- Optimize media formats for different devices
- Reduce latency for a better user experience

14 HANDLING SUDDEN UPLOAD SPIKES



- S3 and CloudFront automatically scale
- Use SQS to buffer processing tasks
- Use Auto Scaling for container workers
- Set concurrency limits
- Monitor queue depth and processing time
- Design for backpressure and graceful degradation

★ KEY TAKEAWAY

A scalable media platform separates storage, processing, and delivery, uses asynchronous workflows, and is designed to handle massive scale, global users, and unexpected traffic spikes.

Better
Engineers
Brighter
Careers

@veriqta

veriqta

veriqta

veriqta

@veriqta

@veriqta

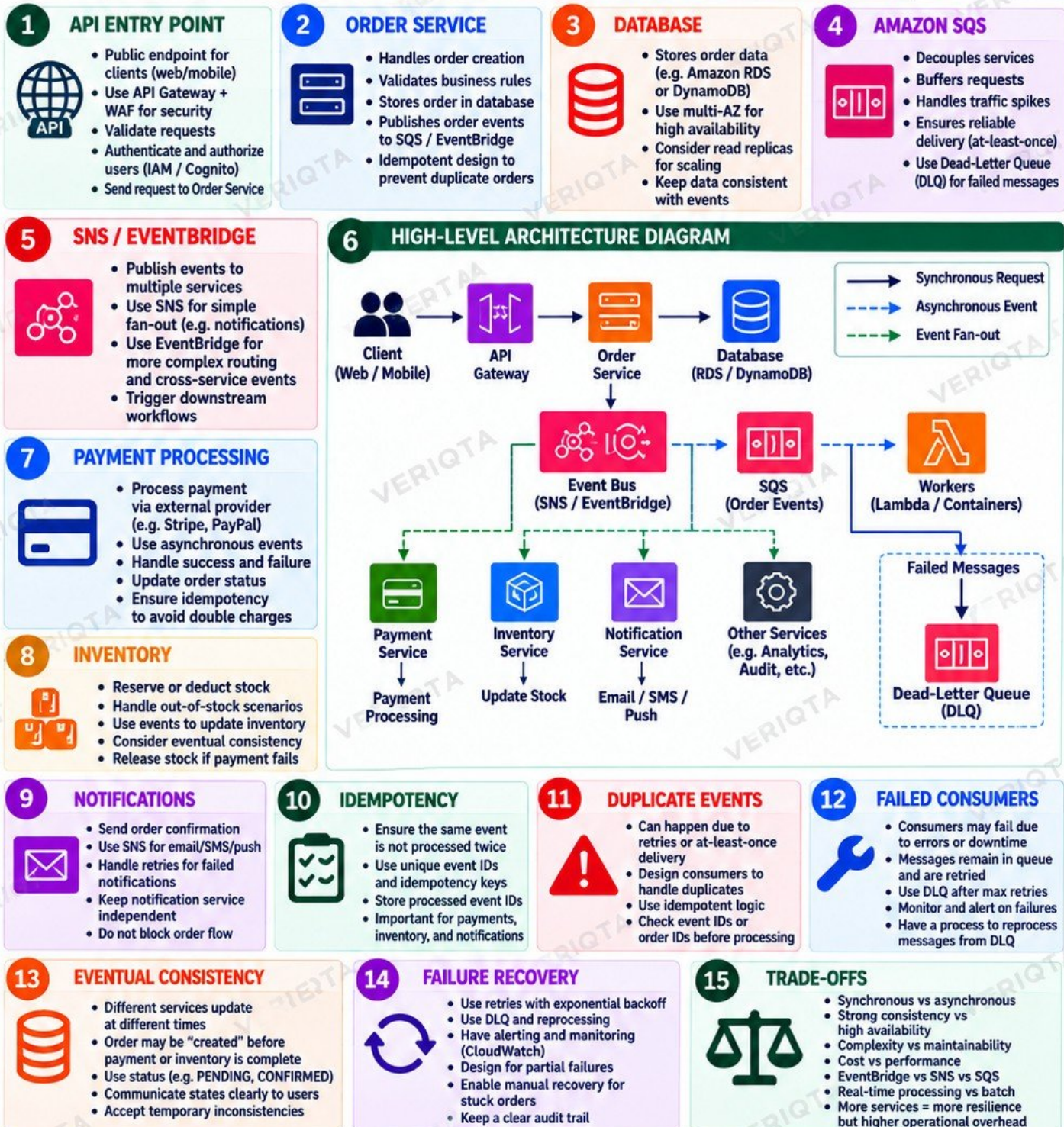
VERIQTA
LEARN | BUILD | GROW

Same
Better
Stronger

DESIGN INTERVIEW, EVENT-DRIVEN ORDER SYSTEM

Build
Design
Scale
Succeed

Design a reliable, scalable, and resilient order system using event-driven architecture.



FINAL MOCK INTERVIEW

DESIGN A PRODUCTION AWS PLATFORM

Apply everything you've learned to design a real-world, scalable, secure, and resilient system.

1 RECEIVE AN AMBIGUOUS DESIGN PROBLEM



- Listen carefully
- Understand the goal
- Identify what's not clear
- Avoid making assumptions

2 ASK CLARIFYING QUESTIONS



- Who are the users?
- What are the key features?
- What are the constraints?
- What is the expected scale?
- What are the success criteria?

3 DEFINE FUNCTIONAL REQUIREMENTS



- Core features
- User actions
- Data to store and retrieve
- Integrations with other systems

4 DEFINE NON-FUNCTIONAL REQUIREMENTS



- Scalability
- High availability
- Low latency
- Security
- Durability
- Cost efficiency
- Compliance

5 ESTIMATE TRAFFIC AND STORAGE



- Daily and monthly users
- Requests per second (RPS)
- Read/write ratios
- Data growth
- Storage and bandwidth
- Peak traffic and scaling needs

6 DRAW THE HIGH-LEVEL ARCHITECTURE



- Show the main components
- Show how they connect
- Keep it simple and clear
- Explain the data flow

7 SELECT AWS SERVICES



- Choose the right services
- Justify each choice
- Consider multiple options
- Explain trade-offs

8 EXPLAIN EVERY MAJOR CHOICE



- Why this service?
- Why this architecture?
- How it meets the requirements?
- What are the alternatives?

9 IDENTIFY BOTTLENECKS



- Compute bottlenecks
- Database bottlenecks
- Network bottlenecks
- Storage bottlenecks
- Third-party dependencies

10 SCALE THE ARCHITECTURE



- Horizontal scaling
- Auto Scaling
- Load balancing
- Caching
- Database scaling
- Handling traffic spikes

11 REMOVE SINGLE POINTS OF FAILURE



- Use multiple Availability Zones
- Add redundancy
- Implement automatic failover
- Design for component failures
- Test failure scenarios

12 ADD SECURITY



- Use IAM and least privilege
- Encrypt data at rest and in transit
- Use VPC, Security Groups, WAF
- Protect against common threats
- Follow security best practices

13 ADD OBSERVABILITY



- Use CloudWatch for metrics and logs
- Set up alarms and dashboards
- Use distributed tracing (X-Ray)
- Monitor key business metrics
- Plan for incident response

14 PLAN DISASTER RECOVERY



- Define RTO and RPO
- Use backups and replication
- Consider multi-region (if needed)
- Test recovery procedures
- Document the recovery plan

15 DISCUSS COST



- Estimate service costs
- Consider cost optimization
- Right-size resources
- Use managed services where it makes sense
- Explain cost vs. availability trade-offs

16 IDENTIFY TRADE-OFFS



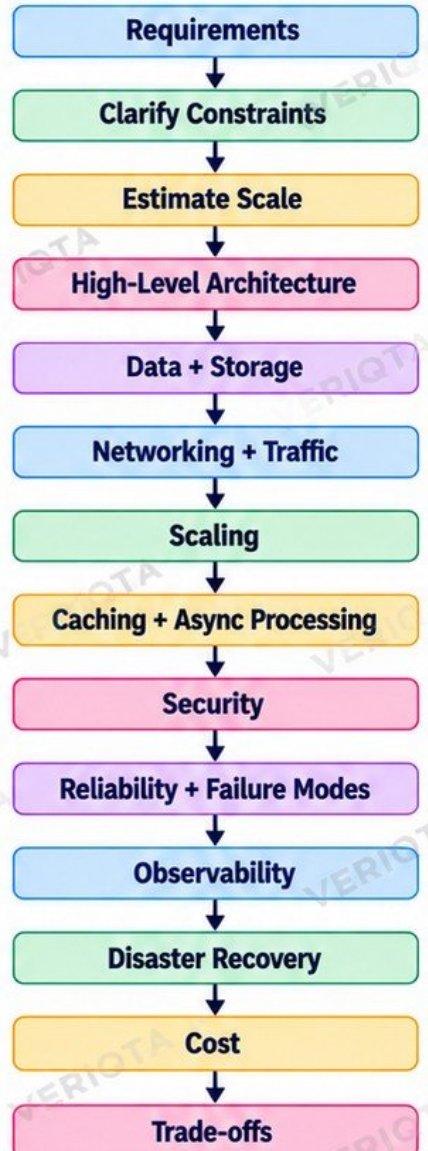
- Cost vs. performance
- Simplicity vs. complexity
- Availability vs. cost
- Managed vs. self-managed
- Short-term vs. long-term
- Explain your reasoning

17 RESPOND TO INTERVIEWER FOLLOW-UP CHANGES



- Handle new requirements
- Explain how your architecture changes
- Discuss the impact on cost, scale, and reliability
- Stay calm and think out loud

FINAL INTERVIEW MENTAL MODEL



"A great architecture is not just about using the right services, but making the right trade-offs for the real world."

— VERIQTA